

Introduction to Spark

🔔 Overview

Teaching: 40 min

Exercises: 0 min

Questions

• How do you program using Spark?

Objectives

• Learn how to use PySpark to write Spark-based programs.

Spark Overview

A Spark program typically follows a simple paradigm:

- A *driver* is the main program.
- One or more workers, called *executors*, run code sent to them by the driver on their partitions of the RDD which is distributed across the cluster.
- Results are then sent back to the driver for aggregation or compilation.

Remember that an RDD is a Resilient Distributed Data set, which is essentially a distributed collection of items.

Essentially the driver program creates one or more RDDs, applies operations to transform the RDD, then invokes some action on the transformed RDD.

These steps are outlined as follows:

1. Define one or more RDDs either through accessing data stored on disk (HDFS, Cassandra, HBase, Local Disk), parallelizing some collection in memory, transforming an existing RDD, or by caching or saving.
2. Invoke operations on the RDD by passing closures (functions) to each element of the RDD. Spark offers over 80 high level operators beyond Map and Reduce.
3. Use the resulting RDDs with actions (e.g. count, collect, save, etc.). Actions kick off the computing on the cluster.

When Spark runs a closure on a worker, any variables used in the closure are copied to that node, but are maintained within the local scope of that closure.

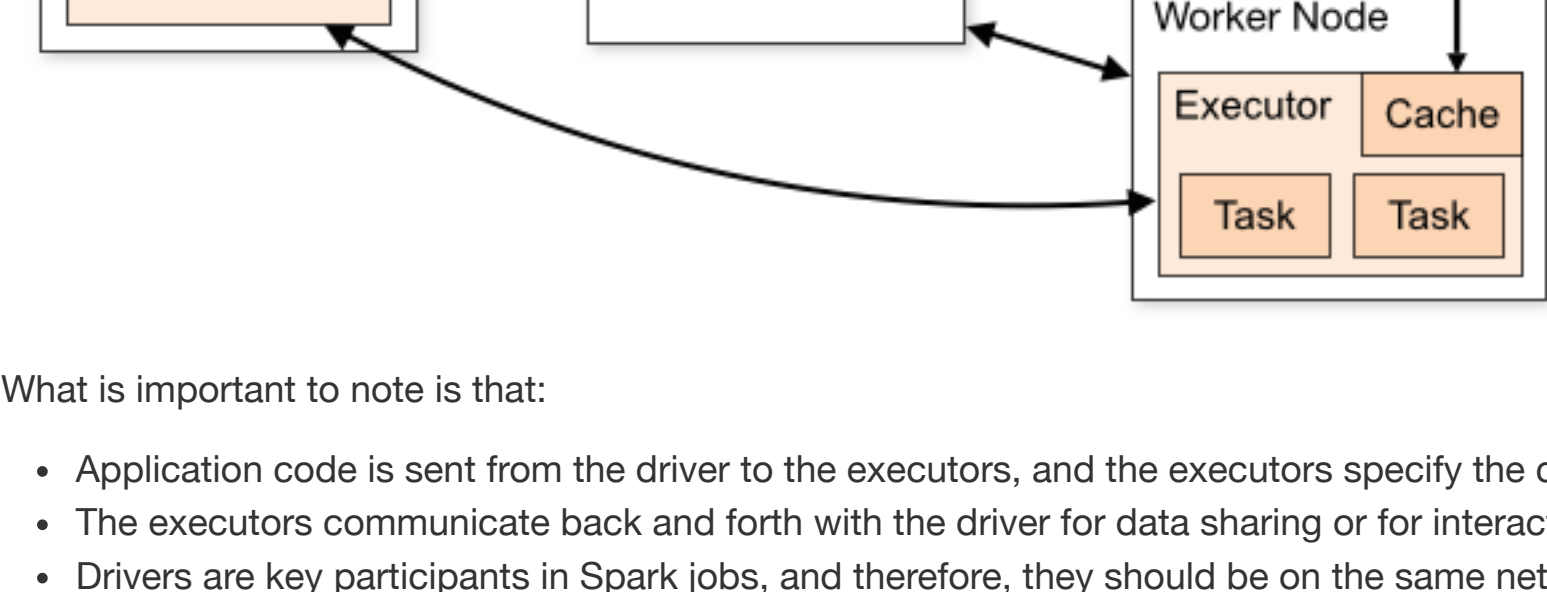
Spark provides two types of shared variables that can be interacted with by all workers in a restricted fashion:

- *Broadcast variables* are distributed to all workers, but are read-only. These variables can be used as lookup tables or stopword lists.
- *Accumulators* are variables that workers can “add” to using associative operations and are typically used as counters.

Spark Execution

Spark applications are run as independent sets of processes, coordinated by a SparkContext in the driver program. The context will connect to some cluster manager (e.g. YARN) which allocates system resources.

Each worker in the cluster is managed by an executor, which is in turn managed by the SparkContext. The executor manages computation as well as storage and caching on each machine.



What is important to note is that:

- Application code is sent from the driver to the executors, and the executors specify the context and the various tasks to be run.
- The executors communicate back and forth with the driver for data sharing or for interaction.
- Drivers are key participants in Spark jobs, and therefore, they should be on the same network as the cluster.

This is different from Hadoop code, where you might submit a job from anywhere to the JobTracker, which then handles the execution on the cluster.

MapReduce with Spark

To start using Spark, we have to create an RDD. The SparkContext provides a number of methods to do this. We will use the `textFile` method, which reads a file and creates an RDD of strings, one for each line in the file.

Create a file called `wordcount_spark.py` with the following code:

```
from pyspark import SparkContext

sc = SparkContext("local", "wordcount")

text = sc.textFile('pg2701.txt')
print(text.take(10))
```

We run this using the PySpark `spark-submit` command as follows:

```
spark-submit wordcount_spark.py
```

```
[ 'The Project Gutenberg eBook of Moby Dick; or The Whale, by Herman Melville', ' ', 'This eBook is for the use of anyone anywhere at no cost and with',
'almost no restrictions whatsoever. You may copy it, give it away or', 're-use it under the terms of the Project Gutenberg License included',
'with this eBook or online at www.gutenberg.org', ' ', ' ', 'Title: Moby Dick; or The Whale', ' ']
```

We use the same splitter function we used previously to split lines correctly. The `flatMap` method applies the function to all elements of the RDD and flattens the results into a single list of words, as follows:

```
words = text.flatMap(splitter)
```

Making these changes, `wordcount_spark.py` now looks like this:

```
from pyspark import SparkContext
import re

# remove any non-words and split lines into separate words
# finally, convert all words to lowercase
def splitter(line):
    line = re.sub(r'[^a-zA-Z]+', ' ', line)
    return map(str.lower, re.split(r'\W+', line))

if __name__ == '__main__':
    sc = SparkContext("local", "wordcount")

    text = sc.textFile('pg2701.txt')
    words = text.flatMap(splitter)
    print(words.take(10))
```

After running this, words will contain the individual words:

```
[ 'the', 'project', 'gutenberg', 'ebook', 'of', 'moby', 'dick', 'or', 'the', 'whale']
```

Now we need to perform the mapping step. This is simply the case of applying the function `lambda x: (x,1)` to each element:

```
words_mapped = words.map(lambda x: (x,1))
```

Our `wordcount_spark.py` program now looks like this:

```
from pyspark import SparkContext
import re

# remove any non-words and split lines into separate words
# finally, convert all words to lowercase
def splitter(line):
    line = re.sub(r'[^a-zA-Z]+', ' ', line)
    return map(str.lower, re.split(r'\W+', line))

if __name__ == '__main__':
    sc = SparkContext("local", "wordcount")

    text = sc.textFile('pg2701.txt')
    words = text.flatMap(splitter)
    words_mapped = words.map(lambda x: (x,1))
    print(words_mapped.take(10))
```

Running the program results in the mapped RDD:

```
[ ('the', 1), ('project', 1), ('gutenberg', 1), ('ebook', 1), ('of', 1), ('moby', 1), ('dick', 1), ('or', 1), ('the', 1), ('whale', 1)]
```

Next, the shuffling step is performed using the `sortByKey` method, which does not require any arguments:

```
sorted_map = words_mapped.sortByKey()
```

Adding this to our `wordcount_spark.py` program results in:

```
from pyspark import SparkContext
import re

# remove any non-words and split lines into separate words
# finally, convert all words to lowercase
def splitter(line):
    line = re.sub(r'[^a-zA-Z]+', ' ', line)
    return map(str.lower, re.split(r'\W+', line))

if __name__ == '__main__':
    sc = SparkContext("local", "wordcount")

    text = sc.textFile('pg2701.txt')
    words = text.flatMap(splitter)
    words_mapped = words.map(lambda x: (x,1))
    sorted_map = words_mapped.sortByKey()
    print(sorted_map.take(10))
```

The output that is generated is shown below. Empty strings are generated by the splitter function for blank lines. Since these sort lexically first, we now see them in the output.

```
[ ('', 1), ('', 1), ('', 1), ('', 1), ('', 1), ('', 1), ('', 1), ('', 1), ('', 1), ('', 1), ('', 1)]
```

For the reduce step, we use the `reduceByKey` method to apply a supplied function to merge values for each key. In this case, the add function will perform a sum. We need to import the add operator in order to be able to use it as follows:

```
counts = sorted_map.reduceByKey(add)
```

Now `wordcount_spark.py` looks like:

```
from pyspark import SparkContext
from operator import add # Required for reduceByKey
import re

# remove any non-words and split lines into separate words
# finally, convert all words to lowercase
def splitter(line):
    line = re.sub(r'[^a-zA-Z]+', ' ', line)
    return map(str.lower, re.split(r'\W+', line))

if __name__ == '__main__':
    sc = SparkContext("local", "wordcount")

    text = sc.textFile('pg2701.txt')
    words = text.flatMap(splitter)
    words_mapped = words.map(lambda x: (x,1))
    sorted_map = words_mapped.sortByKey()
    counts = sorted_map.reduceByKey(add)
    print(counts.take(10))
```

Here is the output after this step:

```
[ ('', 3235), ('dunfermline', 1), ('heedful', 6), ('circle', 24), ('divers', 4), ('riotously', 1), ('patrolled', 1), ('mad', 37), ('lapsed', 1),
('tents', 3)]
```

This is very close to the final result, since we have a count of each of the words. We can use the `max` method to find the word with the maximum number of occurrences. Here is the final version of `wordcount_spark.py`:

```
from pyspark import SparkContext
from operator import add
import re

# remove any non-words and split lines into separate words
# finally, convert all words to lowercase
def splitter(line):
    line = re.sub(r'[^a-zA-Z]+', ' ', line)
    return map(str.lower, re.split(r'\W+', line))

if __name__ == '__main__':
    sc = SparkContext("local", "wordcount")

    text = sc.textFile('pg2701.txt')
    words = text.flatMap(splitter)
    words_mapped = words.map(lambda x: (x,1))
    sorted_map = words_mapped.sortByKey()
    counts = sorted_map.reduceByKey(add)
    print(counts.max(lambda x: x[1]))
```

Here is the final output:

```
[ ('the', 14620)]
```

Parallelizing with Spark

Spark also provides the `parallelize` method which distributes a local Python collection to form an RDD (obviously a cluster is required to obtain true parallelism).

The following example shows how we can calculate the number of primes in a certain range of numbers. First, we define a function to check if a number is prime. This requires checking if it is divisible by all odd numbers up to the square root.

```
def isprime(n):
    """
    check if integer n is a prime
    """
    # make sure n is a positive integer
    n = abs(int(n))
    # 0 and 1 are not primes
    if n < 2:
        return False
    # 2 is the only even prime number
    if n == 2:
        return True
    # all other even numbers are not primes
    if not n & 1:
        return False
    # range starts with 3 and only needs to go up the square root of n
    # for all odd numbers
    for x in range(3, int(n**0.5)+1, 2):
        if n % x == 0:
            return False
    return True
```

Now we can create an RDD comprising all numbers from 0 to n (in this case n = 1000000) using the code:

```
# Create an RDD of numbers from 0 to 1,000,000
nums = sc.parallelize(range(1000000))
```

Finally, we use the `filter` method to apply the function to each value, returning an RDD containing only values that evaluate to True. We can then count these to determine the number of primes.

```
# Compute the number of primes in the RDD
print(nums.filter(isprime).count())
```

Here is the final version of the `primes.py` program:

```
from pyspark import SparkContext

def isprime(n):
    """
    check if integer n is a prime
    """
    # make sure n is a positive integer
    n = abs(int(n))
    # 0 and 1 are not primes
    if n < 2:
        return False
    # 2 is the only even prime number
    if n == 2:
        return True
    # all other even numbers are not primes
    if not n & 1:
        return False
    # range starts with 3 and only needs to go up the square root of n
    # for all odd numbers
    for x in range(3, int(n**0.5)+1, 2):
        if n % x == 0:
            return False
    return True

if __name__ == '__main__':
    sc = SparkContext("local", "primes")
    # Create an RDD of numbers from 0 to 1,000,000
    nums = sc.parallelize(range(1000000))
    # Compute the number of primes in the RDD
    print(nums.filter(isprime).count())
```

Running this program generates the correct answer:

```
[ 78498]
```

References

- Benjamin Bengfort, [Getting Started with Spark \(in Python\)](#)
- A Hands-on Introduction to MapReduce in Python
- Lucas Allen, [Spark Dataframes and MLlib](#)

🔔 Key Points

- Spark defines an API for distributed computing using distributed data sets.
- A driver program coordinates the overall computation.
- An executor is a process that runs computations and stores data.
- Application code is sent to the executors.
- Tasks are sent to the executors to run.